UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

Sistemas Distribuidos

Clase 06 — Storage distribuido: almacenamiento permanente, acceso concurrente y rendimiento bajo fallas

Departamento de Informática

Ingeniería Informática – Comisión única – Pilar

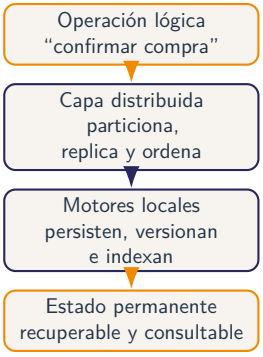
Equipo docente – 2026

Pregunta central: ¿qué resuelve el storage distribuido?

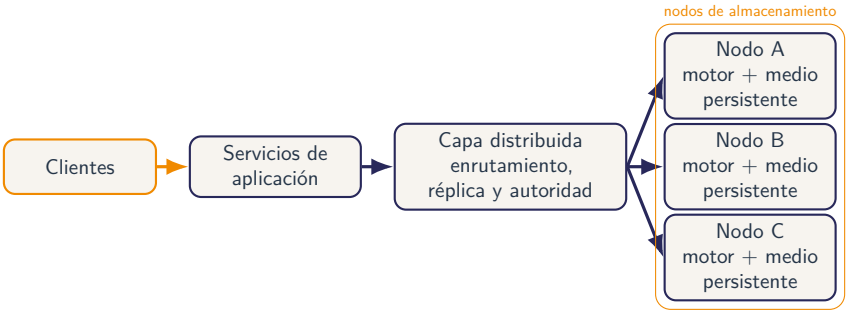
NÚCLEO

En esta clase, **storage distribuido** significa **almacenamiento distribuido**: nodos y protocolos que convierten operaciones en estado permanente, recuperable y consultable aun cuando ocurren fallas.

- Conserva decisiones después de un reinicio.
- Coordina lecturas y escrituras simultáneas.
- Encuentra datos con latencia predecible.
- Absorbe carga sin acumular trabajo sin límite.



Dónde se ubica el storage dentro del sistema distribuido



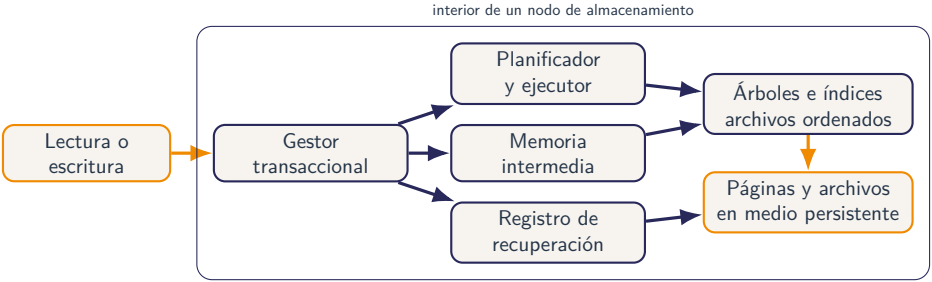
Capa distribuida

Ubica y replica los datos; además, decide qué nodo tiene autoridad para confirmar cada operación.

Motor local

Convierte esas decisiones en bytes persistentes, versiones visibles, índices y procedimientos de recuperación.

Qué llamaremos “motor de almacenamiento”



Definición

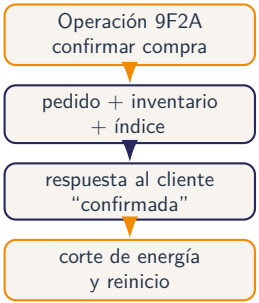
El **motor de almacenamiento** es el software que traduce operaciones lógicas en bytes, versiones, estructuras de acceso y reglas de recuperación sobre un medio persistente.

Caso conductor: CampusShop

NÚCLEO

Una estudiante compra la **última unidad** de un producto del campus. La API (interfaz de programación de aplicaciones) recibe la operación con identificador estable **9F2A** y responde "compra confirmada". Un segundo después, el nodo pierde energía.

- La compra debe existir **exactamente una vez** al reiniciar.
- La unidad descontada no debe reaparecer en el inventario.
- Un reporte largo debe conservar una vista coherente mientras entran nuevas compras.
- La consulta de las últimas 20 compras debe evitar revisar toda la tabla.
- Una carga sostenida no debe crear una cola interna sin límite.



Predicción inicial

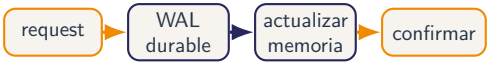
¿Qué evidencia debe sobrevivir para que las cinco afirmaciones anteriores sigan siendo verdaderas?

Camino crítico y patrón de entrada/salida

NÚCLEO

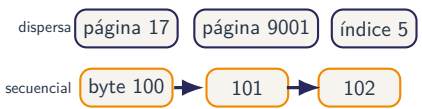
Camino crítico de una escritura

Secuencia de pasos que debe completarse antes de enviar la confirmación al cliente. El paso más lento de esa secuencia condiciona la latencia observada.



E/S aleatoria y secuencial

Aleatoria no significa imprevisible: significa acceder a ubicaciones físicas no contiguas, por ejemplo P17, P9001 e I5. **Secuencial** significa agregar bytes contiguos al final de un archivo o log.



La diferencia de costo depende del dispositivo. Aquí importa la forma del acceso: varias ubicaciones dispersas frente a un append contiguo que puede agruparse.

Árbol B y LSM-tree: dos formas de organizar datos

NÚCLEO

Árbol B (B-tree). Índice ordenado y balanceado cuyos nodos suelen corresponder a páginas. Permite localizar una clave o recorrer un rango con pocas lecturas de páginas.

LSM-tree — *Log-Structured Merge-tree*. En español: **árbol de fusión estructurado como log**. Acumula escrituras en memoria, las vuelca a archivos ordenados e inmutables y luego fusiona esos archivos.

Costo característico

Una escritura puede modificar la página de datos y uno o más índices; el costo aparece en el camino crítico o en mantenimiento posterior.

Qué cambia

Evita reescribir inmediatamente muchas ubicaciones dispersas antes del ACK. El costo reaparece después en flush, compactación, lecturas y espacio.

Idea central

Ninguna estructura elimina trabajo. Cada una decide **cuándo**, **dónde** y **con qué patrón de E/S** se paga.

Vocabulario inicial: compactación, trabajo pendiente y pausa de escritura

NÚCLEO

Término	Significado inicial
Compactación	Proceso de fondo que fusiona archivos ordenados, reduce solapamiento y elimina versiones obsoletas cuando ya es seguro hacerlo.
Trabajo pendiente (backlog)	Cantidad de bytes o archivos que esperan ser compactados. Si ingresa trabajo más rápido de lo que se procesa, el backlog crece.
Regulación de escrituras	Mecanismo de admisión que retrasa, bloquea o limita nuevas solicitudes para que la cola interna no crezca sin límite.
Pausa de escritura (write stall)	Caso fuerte de regulación: el motor hace esperar a los escritores o deja de admitir temporalmente nuevas escrituras hasta recuperar capacidad.

Se regula la **solicitud de escritura**: el hilo puede esperar, recibir un retardo, consumir menos permisos de admisión o, según la implementación, obtener una respuesta de sobrecarga.

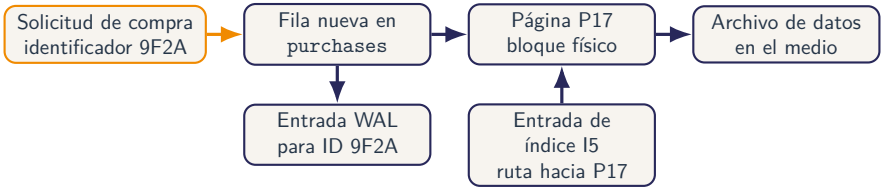
Cinco responsabilidades, no cinco términos aislados

NÚCLEO

Responsabilidad	Pregunta concreta	Mecanismo que estudiaremos
Registrar decisiones	¿Cómo sobrevive una compra ya confirmada?	WAL y recuperación
Controlar concurrencia	¿Qué puede observar un reporte mientras cambian los datos?	MVCC e instantáneas lógicas
Elegir una ruta física	¿Cómo encontrar 20 filas entre un millón?	Árbol B, planificador y `EXPLAIN`
Optimizar escrituras	¿Cómo agrupar escrituras y evitar accesos dispersos inmediatos?	Árbol B o LSM-tree
Proteger capacidad	¿Qué ocurre si la compactación no alcanza el ritmo de ingreso?	Trabajo pendiente, regulación y pausa de escritura

Ningún mecanismo constituye por sí solo el almacenamiento completo. Cada uno cubre una responsabilidad, ofrece una garantía parcial y desplaza un costo.

Del dato lógico a los objetos físicos



Qué significa 9F2A

Es un **identificador estable de la operación lógica**. Si el cliente reintenta, vuelve a enviar 9F2A para que el servidor reconozca la misma compra.

Qué no significa

No es el contenido de la fila ni una posición física. P17 e I5 son etiquetas didácticas para una página de datos y una entrada de índice.

Vocabulario previo I: confirmar y convertir algo en “durable”

NÚCLEO

Término	Significado en esta clase
COMMIT	Decisión del motor de aceptar definitivamente los efectos de una transacción. El COMMIT puede estar decidido antes de que todas las páginas hayan sido escritas.
ACK	*Acknowledgement*, o acuse de recibo. Es la respuesta enviada al cliente cuando se cumplió la garantía que el sistema promete.
X durable	La evidencia necesaria para poder reconstruir X después de ocurrida la falla que el modelo declara. “Durable” siempre debe leerse respecto de una falla concreta.
Log durable	Porción del WAL cuyos bytes necesarios para la recuperación ya sobrevivirían a la falla declarada.
Flush durable	Operación de volcado ya concluida que llevó los bytes requeridos del WAL más allá de la frontera durable.

Regla de lectura

Cuando aparezca “X durable”, pregunte: ¿qué falla se considera y qué evidencia de X seguirá disponible después de esa falla?

Vocabulario previo II: escribir y vaciar buffers

Término	Significado en esta clase
`write`/append	Copiar o agregar bytes a una capa del sistema; la llamada puede terminar mientras los datos siguen en memoria o caché.
Flush o volcado	Mover datos acumulados en un buffer hacia una capa inferior del sistema de almacenamiento. Un flush común no implica necesariamente durabilidad.
`fsync`	Solicitar al sistema operativo que sincronice los datos relevantes con el medio persistente, de acuerdo con el hardware y la configuración.
Entrada/salida (E/S)	Transferencia de datos entre memoria, sistema operativo y dispositivo persistente.
Página sucia (dirty page)	Página modificada en memoria que todavía no fue escrita en el archivo de datos.

Pregunta de ingeniería

Cuando alguien diga “ya está escrito”, pregunte: ¿en qué capa terminó la operación y qué falla todavía podría borrar esos bytes?

Vocabulario previo III: fallar y recuperar

NÚCLEO

Término	Significado en esta clase
Caída abrupta (crash)	Pérdida repentina de un proceso, del sistema operativo o de energía. El modelo debe indicar qué capas se pierden.
Recuperación (recovery)	Procedimiento de reinicio que usa la evidencia persistida para reconstruir un estado lógico válido.
REDO	Reaplicación de un cambio confirmado que aún no estaba materializado en su página de datos o índice.
Punto de control (checkpoint)	Punto durable que acota desde qué posición del WAL debe comenzar la recuperación.
LSN	*Log Sequence Number*: número de secuencia que identifica una posición en el WAL; no es un reloj físico.

Estos términos describen cómo el motor transforma bytes persistidos en un estado lógico válido después de una falla.

Vocabulario previo IV: concurrencia y consistencia

NÚCLEO

Término	Significado en esta clase
Versión	Representación de un dato creada por una transacción y visible para determinados lectores.
Snapshot	Instantánea lógica que determina qué transacciones y versiones puede observar una lectura.
Vista coherente	Resultado cuyas filas obedecen la misma regla de visibilidad, sin mezclar momentos arbitrarios.
Serializabilidad	Garantía de que el resultado concurrente equivale a alguna ejecución de las transacciones una detrás de otra.

Vocabulario previo V: consultas y definición del esquema

NÚCLEO

Término	Significado en esta clase
SQL	*Structured Query Language*: lenguaje estructurado de consultas para leer y modificar datos.
DDL	*Data Definition Language*: parte de SQL que crea o modifica estructuras, por ejemplo `CREATE INDEX`.
Planificador	Componente que compara rutas físicas posibles y elige un plan estimado.
`EXPLAIN`	Comando que muestra el plan elegido por el planificador.
`EXPLAIN ANALYZE`	Ejecuta la consulta y agrega mediciones reales al plan.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Repaso ágil: cinco puentes hacia el almacenamiento

Una frase por cada clase previa

Clase 1 — Incertidumbre: si no sabemos qué ocurrió, el almacenamiento conserva evidencia para responder de manera estable.

Clase 2 — RPC (llamada a procedimiento remoto) y reintento: el reintento es seguro solo si identidad, efecto y resultado sobreviven a la falla.

Clase 3 — Replicación: varias copias no indican por sí solas cuáles son durables, actuales ni autorizadas.

Clase 4 — Consenso: el protocolo decide qué debe recordarse; cada nodo debe persistirlo realmente.

Clase 5 — Transacciones: ACID (atomicidad, consistencia, aislamiento y durabilidad) define el contrato; WAL y MVCC implementan parte de ese contrato.

Transición. Hoy preguntamos qué bytes, esperas y estructuras vuelven verdaderas palabras como “COMMIT”, “durable” y “aislado”.

Diagnóstico inicial: cinco situaciones

ACTIVIDAD

Diagnóstico

Al finalizar esta clase debemos ser capaces de describir el comportamiento que se espera y la falla que está considerada.

1. La aplicación mostró “compra confirmada” y enseguida se cortó la energía. ¿Puede desaparecer al reiniciar?
2. Un reporte comenzó a las 12:00 y durante su ejecución entraron nuevas compras. ¿Deben cambiar sus totales a mitad del cálculo?
3. Para mostrar 20 compras entre un millón, ¿es razonable revisar siempre todas las filas?
4. Ingresan 90 megabytes por segundo (MB/s) y se compactan 60 MB/s. ¿Puede el trabajo pendiente permanecer acotado?
5. Tres nodos conservan el dato solo en memoria RAM (memoria de acceso aleatorio). ¿Sobrevive un corte de energía común?

Mapa conceptual: dónde aparece cada mecanismo

NÚCLEO



El hilo conductor será siempre el mismo: problema, mecanismo, garantía, costo y evidencia.

Evidencias de aprendizaje de esta clase

NÚCLEO

Durante la clase

- completar un timeline WAL y justificar cuándo enviar el ACK;
- completar un timeline MVCC y justificar qué versión es visible;
- leer un plan `EXPLAIN` antes y después de crear un índice.

Al cerrar

- diagnosticar trabajo pendiente y una pausa de escritura;
- elegir entre árbol B, LSM-tree o una combinación según la carga;
- defender la decisión con falla, garantía, costo y métrica.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

WAL y recuperación: hacer durable una decisión

Sistemas Distribuidos

CampusShop: qué cambia físicamente al confirmar una compra

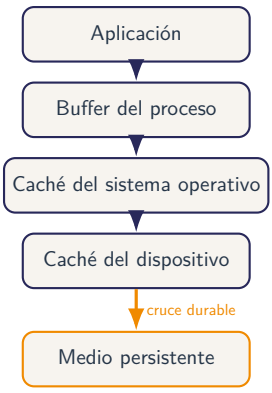
NÚCLEO

La operación con ID **9F2A** inserta una fila en `purchases`, descuenta una unidad en `inventory` y actualiza un índice por estudiante y fecha.

- 9F2A identifica la misma compra lógica si el cliente reintenta.
- Las filas viven lógicamente en tablas.
- El motor modifica páginas en memoria.
- El WAL conserva evidencia para la recuperación.



La frontera durable



Definición

La **frontera durable** es el punto a partir del cual, bajo el modelo de falla declarado, permanece disponible la evidencia necesaria para reconstruir la operación.

No existe una frontera universal

Depende de la falla considerada y de la semántica efectiva de `fsync`, del sistema operativo, del controlador, del dispositivo y de su configuración.

Qué es WAL y por qué se dibuja separado de las páginas

Definición intuitiva

WAL es el registro secuencial del motor. Primero conserva evidencia para reconstruir una decisión; después, las páginas pueden persistirse en un orden físico más conveniente.

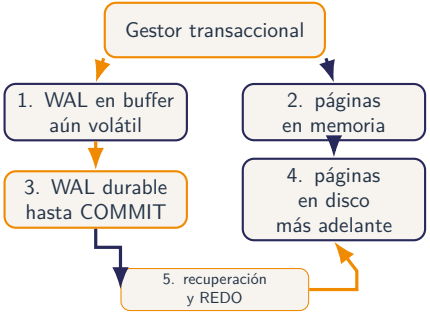
Definición técnica

La evidencia de recuperación debe ser durable antes que la página modificada, y el registro COMMIT debe ser durable antes del ACK.

Por qué se dibuja aparte. WAL y páginas se vuelven durables en momentos distintos, aunque ambos pertenezcan al mismo motor.

Predicción

Si el ACK ya salió y la página persistente conserva la versión anterior, ¿qué evidencia permite reconstruir el cambio?



Dos persistencias distintas: el WAL conserva evidencia; las páginas materializan el estado.

WAL: dos reglas, dos fallas que evitan

NÚCLEO

Regla 1 — log antes que página

Orden: registro WAL → flush durable → página de datos.

- **Propósito:** que toda página persistida tenga evidencia suficiente para ser interpretada o rehecha.
- **Falla evitada:** una página nueva sin registro recuperable del cambio que la produjo.

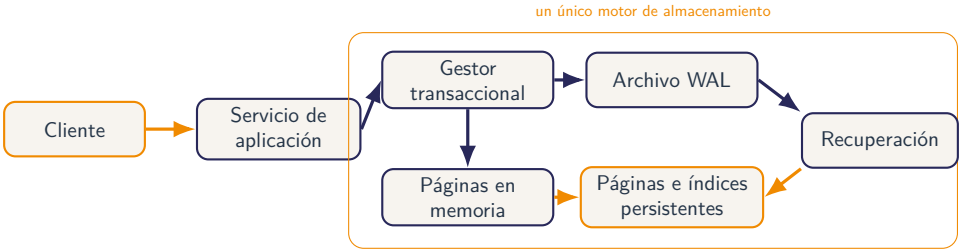
Regla 2 — COMMIT antes que ACK

Orden: registro COMMIT → flush durable → ACK al cliente.

- **Propósito:** que cada confirmación enviada pueda reconstruirse tras un crash.
- **Falla evitada:** mostrar "confirmada" y perder luego la decisión de COMMIT.

Garantía buscada: después de la recuperación aparecen todas las transacciones confirmadas y ninguna transacción incompleta se presenta como confirmada.

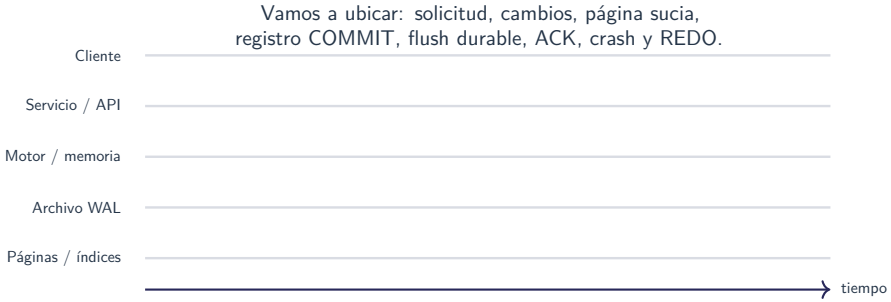
Esquema de funcionamiento del WAL



- **1. Registrar:** el gestor agrega evidencia al WAL.
- **2. Modificar:** las páginas cambian primero en memoria.
- **3. Materializar:** las páginas se escriben después en el dispositivo persistente.
- **4. Recuperar:** tras un crash, el WAL permite completarlas.

Timeline del WAL

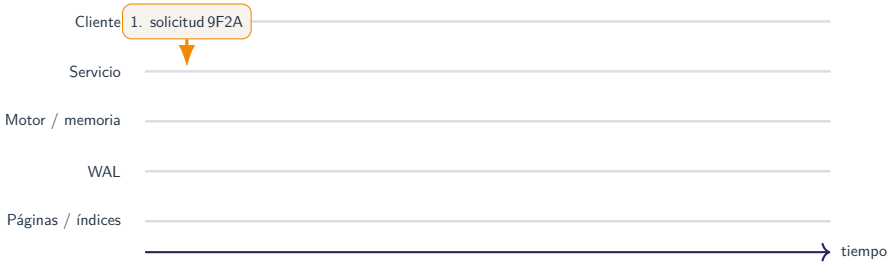
ACTIVIDAD



Predicción

¿Después de qué evento sería legítimo mostrar “compra confirmada” sin exigir que las páginas finales ya estén actualizadas en el dispositivo?

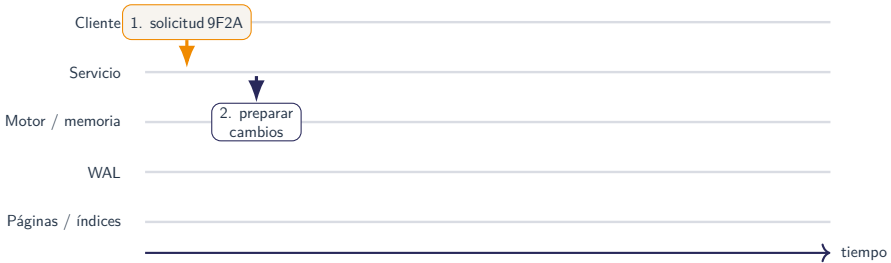
Construcción progresiva del timeline WAL



Paso 1

El cliente envía una intención con identidad estable.

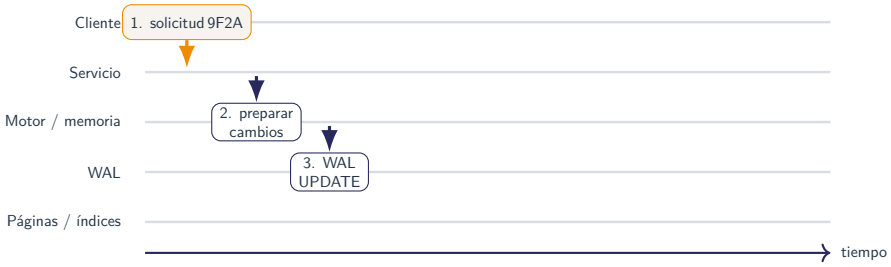
Construcción progresiva del timeline WAL



Paso 2

El motor prepara cambios lógicos en memoria.

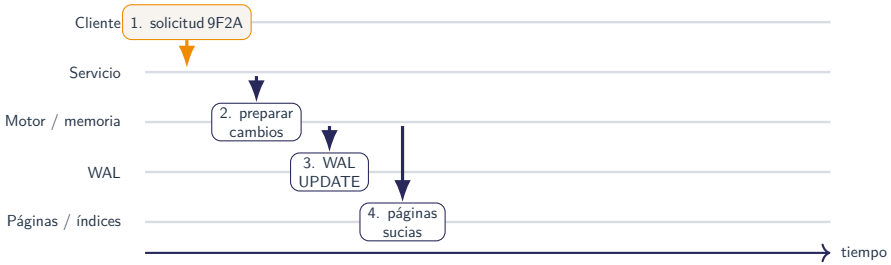
Construcción progresiva del timeline WAL



Paso 3

El WAL describe cómo rehacerlos; todavía puede estar en buffers.

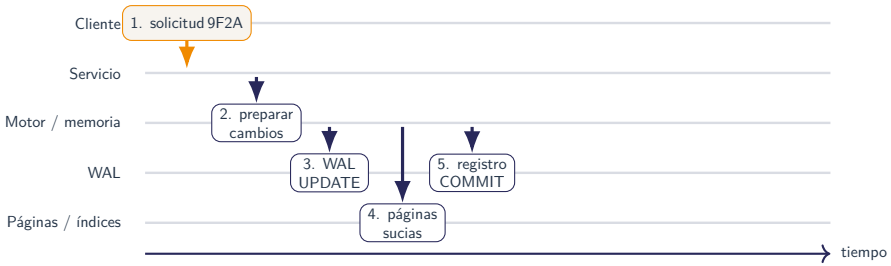
Construcción progresiva del timeline WAL



Paso 4

Las páginas se modifican en memoria y quedan sucias.

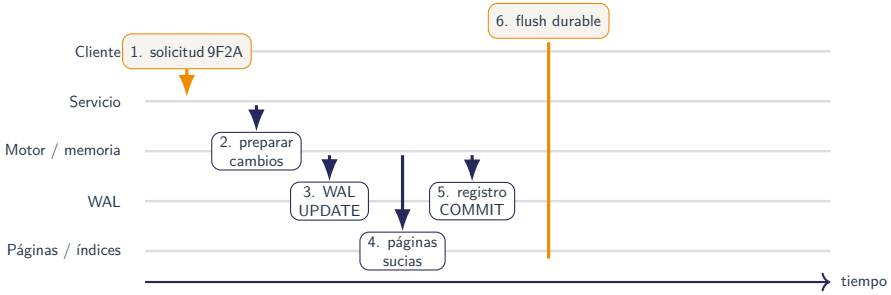
Construcción progresiva del timeline WAL



Paso 5

El registro COMMIT expresa la decisión, pero aún no es durable.

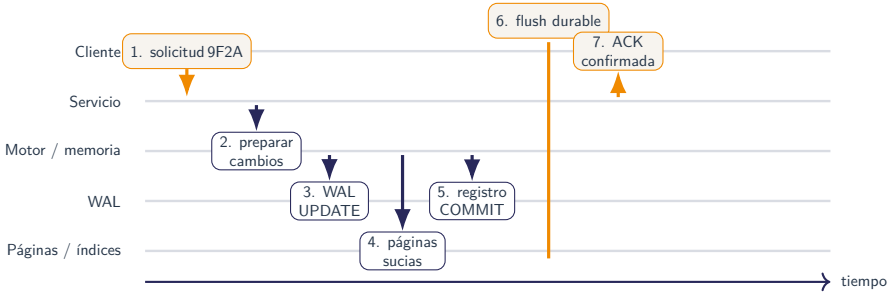
Construcción progresiva del timeline WAL



Paso 6

El flush lleva el WAL relevante más allá de la frontera durable declarada.

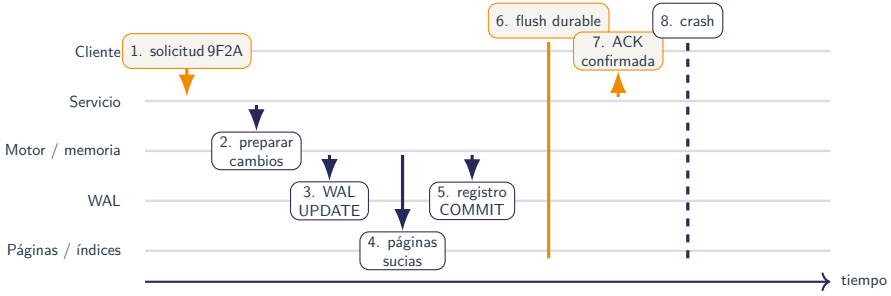
Construcción progresiva del timeline WAL



Paso 7

Ahora el servicio puede confirmar sin esperar que todas las páginas estén escritas.

Construcción progresiva del timeline WAL

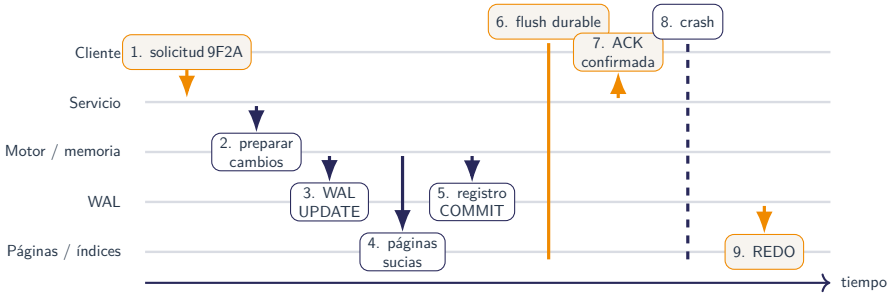


Paso 8

El crash borra memoria y buffers volátiles.

Construcción progresiva del timeline WAL

NÚCLEO



Paso 9

Recuperación lee el WAL y REDO materializa en las páginas los cambios confirmados que faltan.

Criterio de correctitud de la recuperación

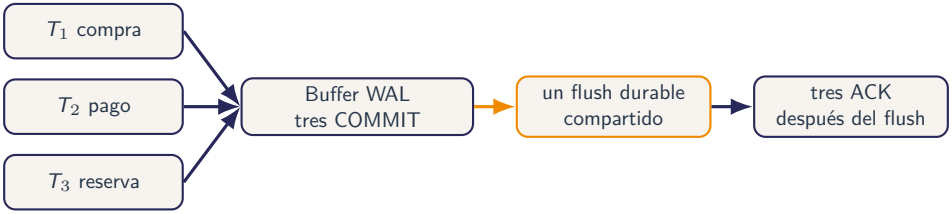
NÚCLEO

Punto del crash	Evidencia durable disponible	Resultado correcto después de recuperar
Antes del COMMIT durable	Puede haber registros de cambio, pero no una decisión durable de COMMIT	La compra no aparece confirmada; el trabajo incompleto se descarta o revierte según el diseño.
Después del COMMIT durable, antes de escribir páginas	Registros de cambio y COMMIT en el WAL durable	REDO reconstruye pedido, stock e índice.
Después del COMMIT y de las páginas	WAL y páginas ya están materializados	La recuperación verifica el estado; repetir REDO no debe producir un segundo efecto lógico.

Los tres resultados son correctos porque aplican el mismo criterio: conservar todas las transacciones confirmadas y no convertir una transacción incompleta en confirmada.

Group commit: compartir el costo sin debilitar la garantía

NÚCLEO



Ganancia: un acceso físico amortiza varios COMMIT. **Costo:** cada transacción puede esperar brevemente.

Garantía: cada ACK sigue saliendo solamente después de que su COMMIT quedó durable.

Checkpoint: un punto durable desde dónde comenzar una recuperación

NÚCLEO



Qué es

Registra suficiente progreso durable para que la recuperación no comience desde el origen del WAL.

- **Más frecuente:** menos WAL por reproducir, pero más entrada/salida y posibles picos.
- **Más espaciado:** operación normal más suave, pero recuperación y retención mayores.

Caso real: PostgreSQL desacopla el COMMIT de las páginas

NÚCLEO

- El WAL relevante debe llegar al almacenamiento estable antes que los cambios en los archivos de datos.
- Tras un crash, la recuperación reproduce lo que indica el WAL para restaurar un estado válido.
- Las páginas persistentes pueden conservar temporalmente una versión anterior al último COMMIT.
- El hardware y la configuración forman parte de la garantía efectiva.



En esta lámina, una **página aún no actualizada** es una página cuyo archivo persistente todavía no incorporó un cambio confirmado. La evidencia durable está en el WAL y REDO permite materializar el cambio al reiniciar.

Fuente / referencia: Documentación oficial de PostgreSQL, sección Write-Ahead Logging.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

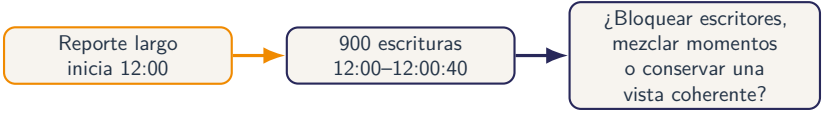
MVCC: versiones para lecturas concurrentes coherentes

Sistemas Distribuidos

Pensemos en ejemplo real: se crea y consulta un reporte y concurrentemente se hacen 900 compras nuevas

NÚCLEO

Un reporte del comedor comienza a las 12:00 y tarda 40 segundos. Durante ese intervalo entran 900 compras. ¿Debe mezclar filas de momentos distintos? ¿Debe bloquear a todos los escritores?



Vocabulario de MVCC

NÚCLEO

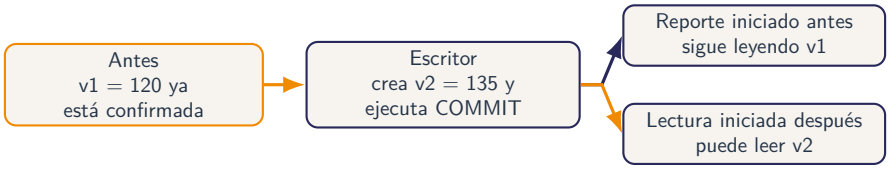
Término	Definición de trabajo
Versión	Representación de una fila creada por una transacción y válida para determinados lectores.
Transacción lectora	Operación que consulta datos bajo una regla de visibilidad.
Transacción escritora	Operación que crea una versión nueva y no borra de inmediato la anterior.
Snapshot	Instantánea lógica que determina qué transacciones confirmadas y qué versiones son visibles.
Visibilidad	Decisión de si una versión pertenece a la historia que ese snapshot puede observar.
Vista coherente	Resultado cuyas filas se seleccionan con la misma regla de visibilidad.
Serializabilidad	Garantía de que la ejecución concurrente equivale a alguna ejecución serial de las transacciones.

Límite de la metáfora

"Instantánea o Fotografía" no significa copiar toda la base. Tampoco implica serializabilidad por sí sola: describe una regla para elegir versiones.

MVCC en una imagen: dos lectores, dos vistas coherentes

NÚCLEO

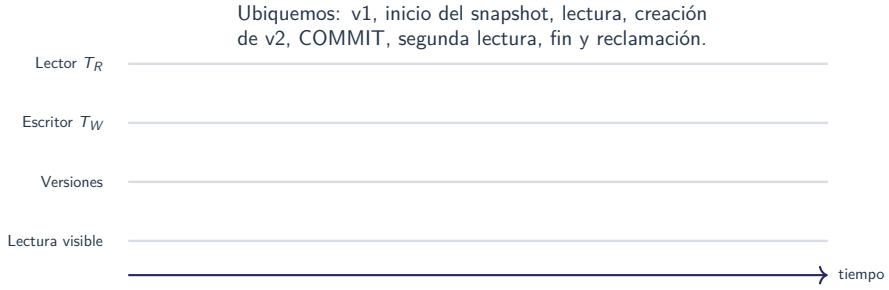


Idea central

MVCC conserva versiones y aplica reglas de visibilidad. Dos lectores pueden observar versiones diferentes sin que el reporte antiguo cambie a mitad de ejecución.

Timeline MVCC: cuatro carriles y términos conocidos

ACTIVIDAD

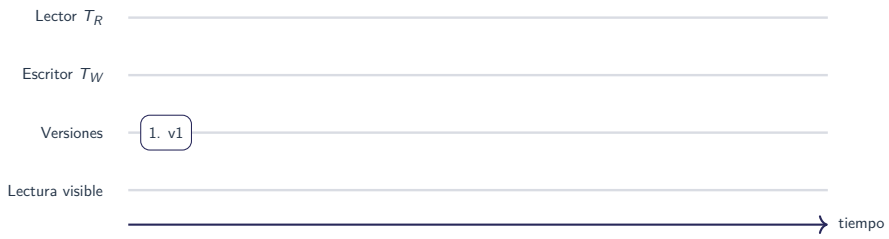


Consideración

Después del COMMIT de una v2, ¿qué lee una segunda consulta dentro del mismo reporte y qué lee un reporte nuevo?

Construcción progresiva del timeline MVCC

NÚCLEO

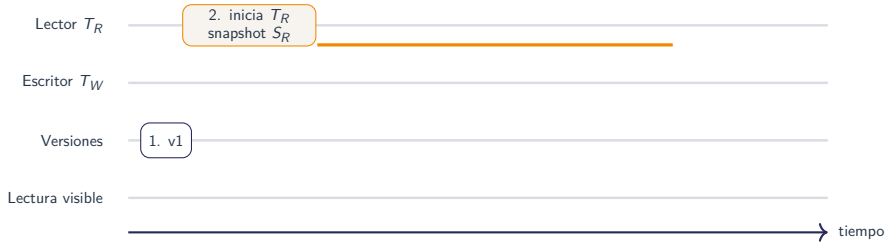


Paso 1

v1 es la versión confirmada existente.

Construcción progresiva del timeline MVCC

NÚCLEO

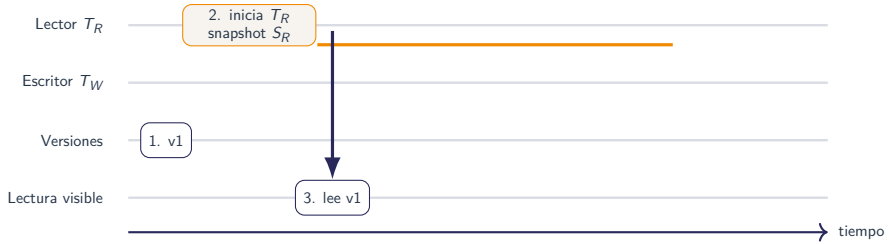


Paso 2

T_R fija una regla de visibilidad: su snapshot.

Construcción progresiva del timeline MVCC

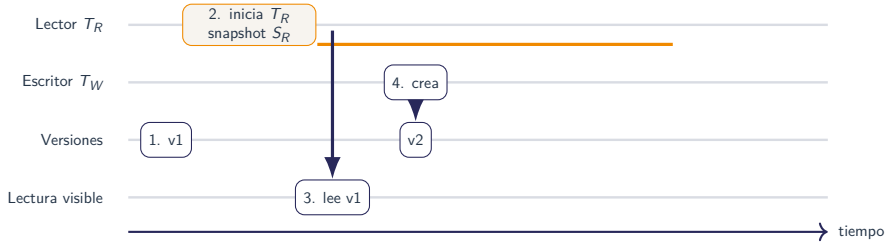
NÚCLEO



Paso 3

La primera lectura elige v1.

Construcción progresiva del timeline MVCC

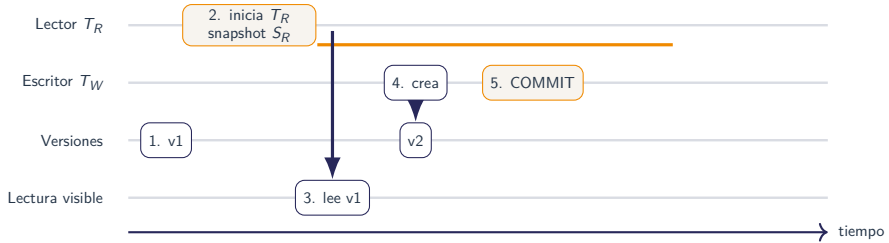


Paso 4

T_W crea v2 sin borrar v1.

Construcción progresiva del timeline MVCC

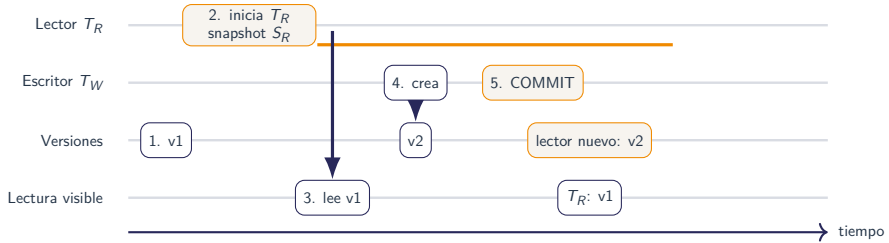
NÚCLEO



Paso 5

v2 confirma y queda disponible para snapshots posteriores.

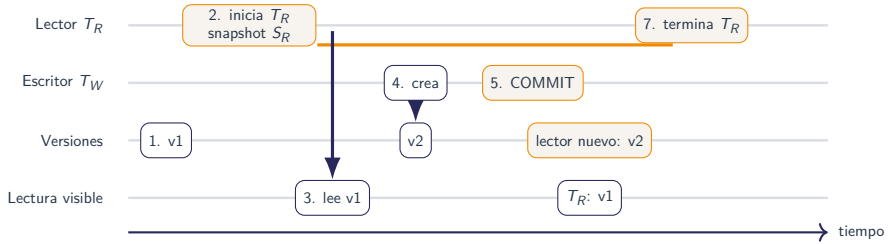
Construcción progresiva del timeline MVCC



Paso 6

T_R mantiene v1; un lector nuevo puede elegir v2.

Construcción progresiva del timeline MVCC

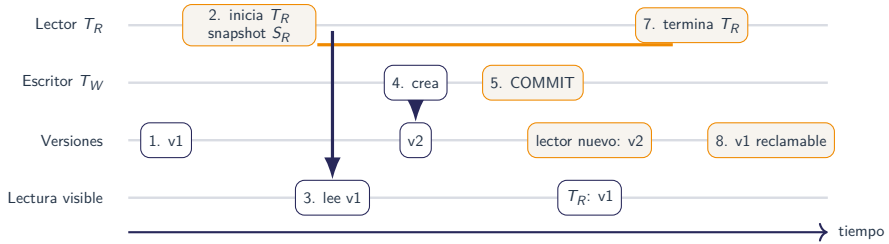


Paso 7

Termina el snapshot largo.

Construcción progresiva del timeline MVCC

NÚCLEO

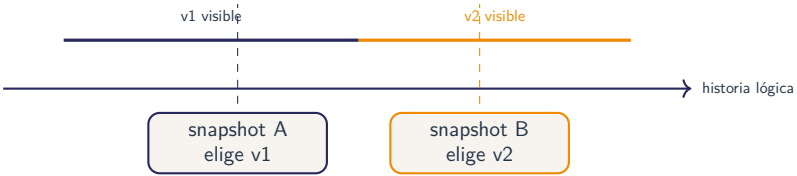


Paso 8

v1 puede limpiarse cuando ninguna regla de visibilidad la necesite.

Regla simplificada de visibilidad

NÚCLEO



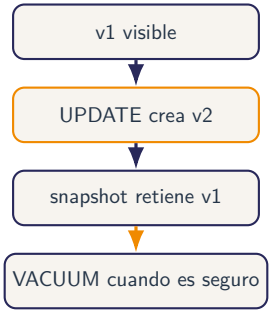
Lectura conceptual

Una versión es visible si su creador pertenece a la historia confirmada que el snapshot puede observar y su reemplazo/eliminación todavía no era visible para ese snapshot.

Caso real: PostgreSQL, versiones de fila y VACUUM

NÚCLEO

- Un `UPDATE` crea una versión nueva de la fila.
- La versión anterior permanece mientras algún snapshot pueda verla.
- `VACUUM` recupera espacio reutilizable cuando la historia deja de ser necesaria.
- Una transacción larga retrasa la limpieza y aumenta bloat, E/S y presión sobre índices.



Fuente / referencia: PostgreSQL, documentación oficial sobre MVCC y routine vacuuming.

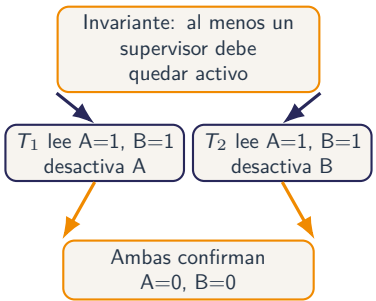
Condición de carrera y write skew bajo MVCC

Condición de carrera

El resultado depende del orden relativo en que operaciones concurrentes leen y modifican estado compartido. Dos ejecuciones válidas por separado pueden producir juntas un resultado inválido.

Write skew o sesgo de escritura

Dos transacciones leen la misma instantánea, actualizan filas diferentes y ambas confirman. No existe un conflicto directo de escritura, pero se viola un invariante que abarca varias filas.



Conclusión

MVCC administra versiones; no define por sí solo serializabilidad. El nivel de aislamiento debe detectar, bloquear o abortar ejecuciones que podrían violar invariantes.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Índices y EXPLAIN: de una versión visible a una ruta eficiente

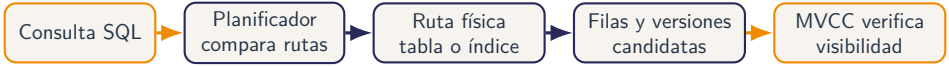
Sistemas Distribuidos

Transición: MVCC decide qué versión; el índice ayuda a encontrarla

NÚCLEO

La pregunta que acabamos de resolver con MVCC
Dadas varias versiones de una fila, ¿cuál puede observar esta transacción según su snapshot?

La nueva pregunta física
¿Cómo localiza el motor las filas candidatas sin recorrer todas las páginas de la tabla?



Puente entre los temas

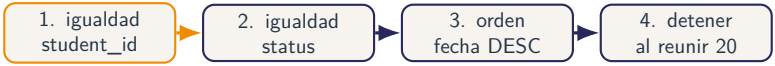
El índice no decide qué versión es correcta: reduce el espacio de búsqueda. MVCC conserva la semántica de la lectura, y `EXPLAIN` permite comprobar qué ruta eligió el planificador.

CampusShop: la consulta define el patrón de acceso

NÚCLEO

Pregunta de producto: mostrar las últimas 20 compras confirmadas de una estudiante.

```
SELECT id, total, confirmed_at
FROM purchases
WHERE student_id = 8421 AND status = 'CONFIRMED'
ORDER BY confirmed_at DESC
LIMIT 20;
```

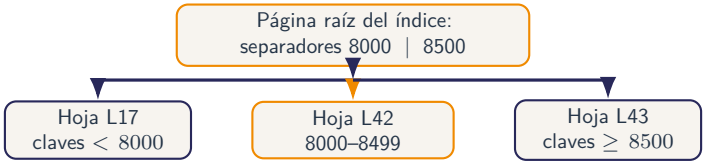


De la consulta a una hipótesis de estructura

La ruta ideal debería ubicar primero `student_id=8421`, conservar solo `status='CONFIRMED'`, recorrer `confirmed_at` en orden descendente y detenerse al obtener 20 filas. Un índice compuesto puede materializar precisamente ese orden.

Árbol B: páginas balanceadas para evitar un recorrido completo

NÚCLEO



Definición conectada con las páginas

Cada nodo del árbol suele almacenarse en una página. Las claves de las páginas internas actúan como separadores; las hojas contienen claves ordenadas y referencias hacia filas o páginas de datos.

Por qué necesita pocos accesos

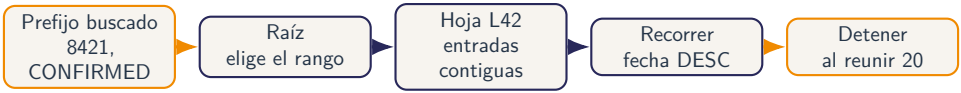
El árbol permanece balanceado y tiene alto *fan-out*: en una página caben muchas claves y referencias. Por eso, aun con millones de filas, la ruta desde la raíz hasta una hoja suele tener pocos niveles.

Costo que no debe ocultarse

Insertar, actualizar o borrar puede modificar páginas del índice y, cuando una página se llena, provocar una división. El árbol acelera determinadas consultas a cambio de más espacio y trabajo de escritura.

CampusShop: el árbol convierte el predicado en una ruta

NÚCLEO



Entradas de la hoja, ya ordenadas

```
(8421, CONFIRMED, 12:05) -> P17
(8421, CONFIRMED, 11:58) -> P81
  ⋮
```

Comprobación final
Las referencias llevan a versiones candidatas. MVCC verifica cuáles son visibles para el snapshot de la consulta antes de devolverlas.

Aporte y transición. El árbol B reduce páginas candidatas, conserva el orden de la clave compuesta y permite detener el recorrido temprano. Todavía falta demostrar que el planificador lo elija: `EXPLAIN` mostrará la ruta y `EXPLAIN ANALYZE` permitirá contrastarla con filas, buffers y tiempos observados.

Tener un índice no garantiza usarlo: EXPLAIN muestra la decisión

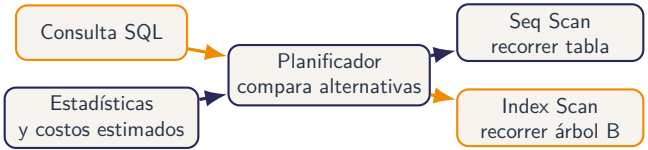
NÚCLEO

EXPLAIN

Muestra el plan elegido y sus **estimaciones**: forma de acceso, filtros, orden, costo y cantidad esperada de filas.

EXPLAIN ANALYZE

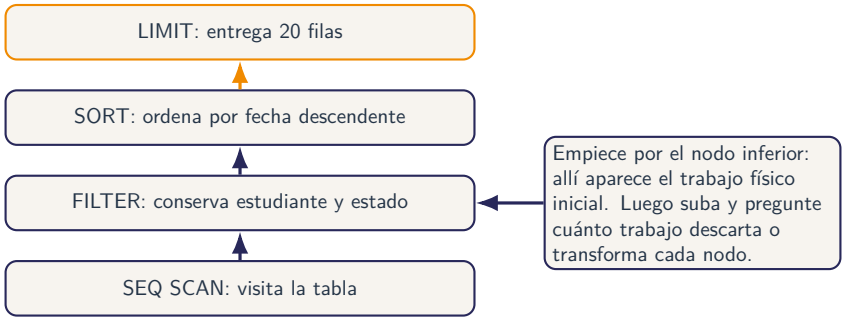
Ejecuta la consulta y agrega **mediciones reales**: tiempos, filas, repeticiones y, con `BUFFERS`, entrada/salida observada.



La estructura disponible es solo una alternativa. La evidencia empieza cuando observamos qué plan fue elegido, cuánto trabajo realizó y si las estimaciones coincidieron con la ejecución.

Cómo leer un plan: de abajo hacia arriba

NÚCLEO



Sin índice: mucho trabajo para encontrar 20 filas


```
Limit (actual time=12.55..12.58 rows=20)
  -> Sort (rows=20)
    Sort Key: confirmed_at DESC
    -> Seq Scan on purchases
      Filter: student_id=8421
      Rows Removed by Filter: 9980
      Buffers: shared hit=840
Execution Time: 12.84 ms
```

Tres evidencias

- Seq Scan.
- 9980 filas descartadas.
- 840 buffers tocados.

Interpretación

El motor recorre, filtra y ordena un conjunto mucho mayor que las 20 filas finales.

Con índice compuesto: el plan hace menos trabajo

NÚCLEO

```
Limit (actual time=0.20..0.24 rows=20)
  -> Index Scan using
    idx_purchase_student_status_date
    on purchases
    Index Cond: student_id=8421
      AND status='CONFIRMED'
    Buffers: shared hit=5
Execution Time: 0.27 ms
```

Qué cambió

- Se tocaron 5 buffers.
- Se obtuvieron 20 filas útiles.
- No hubo ordenamiento separado.
- LIMIT detuvo el recorrido temprano.

DDL utilizado

```
CREATE INDEX ... ON purchases(student_id, status,
confirmed_at DESC);
```

Comparar planes sin convertir los números en una promesa universal

NÚCLEO

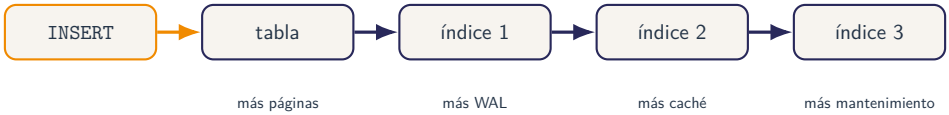
Evidencia	Sin índice	Con índice
Acceso	Seq Scan	Index Scan
Filas descartadas	9980	0 en el ejemplo
Buffers	840	5
Sort explícito	Sí	No
Tiempo ilustrativo	12.84 ms	0.27 ms

Cuidado

Los valores dependen de datos, estadísticas, caché, selectividad y hardware. La lección es aprender a leer evidencia, no memorizar una cifra.

El costo del índice aparece en cada escritura

NÚCLEO



Regla de ingeniería

No se agrega un índice porque "parece útil". Se formula una hipótesis, se observa el plan y se mide la carga de trabajo real.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

LSM-tree: escrituras rápidas, organización posterior

Sistemas Distribuidos

Qué es una LSM-tree y por qué aparece

Definición

Una LSM-tree acumula cambios en una estructura ordenada en memoria. Cuando se llena, la vuelca como una SSTable (*Sorted String Table*: archivo ordenado e inmutable de pares clave–valor). Más tarde fusiona varias SSTables mediante compactación.

En el **camino crítico** de la solicitud, el motor evita actualizar de inmediato muchas páginas dispersas. Registra la escritura y modifica memoria; el trabajo de reorganización física queda para flush y compactación en segundo plano.

No elimina costo

Desplaza trabajo: puede aumentar bytes reescritos, archivos consultados, espacio temporal y latencia cuando el mantenimiento queda atrás.



Vocabulario LSM antes de hablar de compactación

NÚCLEO

Término	Significado
Memtable	Tabla ordenada en memoria que recibe las escrituras recientes.
Flush	Volcado de una memtable inmutable a un archivo persistente.
SSTable	*Sorted String Table*: archivo ordenado e inmutable de pares clave–valor.
Run	Secuencia o archivo ya ordenado que puede participar en una fusión.
Nivel L0, L1, L2	L0 es el primer nivel persistente que recibe archivos del flush; L1, L2 y siguientes organizan SSTables progresivamente mayores según la política de compactación.
Marca de borrado (tombstone)	Registro que representa una eliminación y debe conservarse hasta que sea seguro olvidar versiones anteriores.

Vocabulario LSM: mantenimiento y protección

NÚCLEO

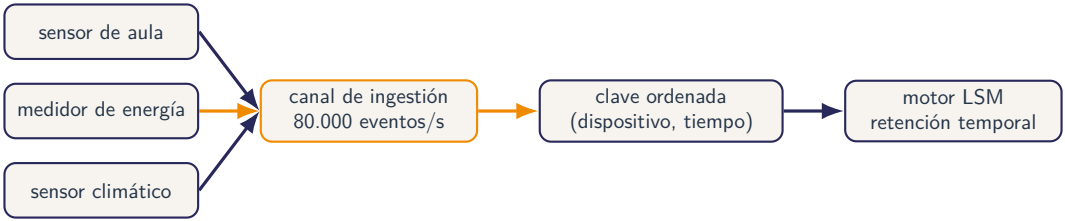
Término	Significado
Fusión ordenada (merge)	Recorre varias secuencias ya ordenadas y produce otra secuencia ordenada, eligiendo en cada paso la menor clave disponible.
Compactación	Fusión ordenada que además reduce solapamiento y elimina versiones obsoletas o tombstones cuando es seguro.
Trabajo pendiente (backlog)	Bytes o SSTables que esperan compactación. Crece cuando la tasa de ingreso supera sostenidamente la capacidad de mantenimiento.
Regulación de admisión	El motor introduce espera, bloquea hilos o limita permisos para reducir la tasa de nuevas escrituras.
Pausa de escritura (write stall)	Regulación fuerte en la que una solicitud de escritura no avanza hasta que el trabajo de fondo reduce el backlog o se libera un recurso.

Se puede retrasar o bloquear la **solicitud** que intenta generar más trabajo.

Caso contrastante: telemetría del campus

NÚCLEO

Sensores de aulas, energía y ocupación producen 80.000 eventos por segundo. Las consultas recorren un dispositivo y una ventana temporal.



- Tasa de escritura alta y sostenida.
- Claves ordenables por dispositivo y tiempo.
- Retención y borrado por antigüedad.
- Búsquedas puntuales y rangos temporales.

Ruta de escritura LSM: de la solicitud a una SSTable L0

NÚCLEO



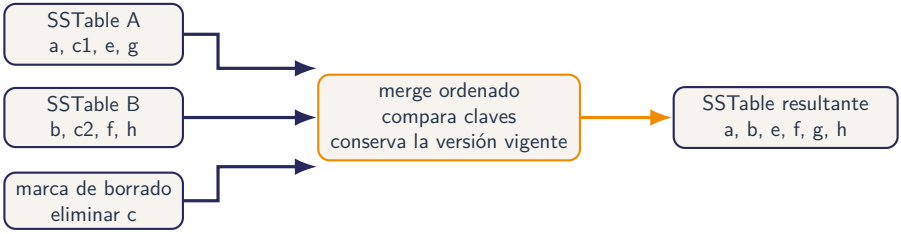
Qué ocurre antes del ACK

Según la garantía de durabilidad, la escritura se registra en el WAL y se incorpora a la memtable. El flush de la memtable inmutable hacia una SSTable L0 puede ocurrir después y no tiene por qué bloquear esa misma solicitud.

Qué queda para después

L0 puede acumular archivos solapados. La compactación posterior reorganiza esos archivos y los desplaza hacia niveles más estables.

Compactación: merge ordenado y eliminación segura



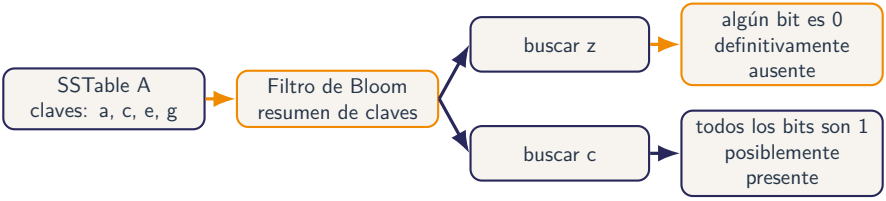
Qué se paga

La compactación lee y reescribe datos, utiliza espacio temporal, compite por E/S y puede aumentar el percentil 99 de latencia.

Filtro de Bloom: decidir qué SSTables se pueden omitir

NÚCLEO

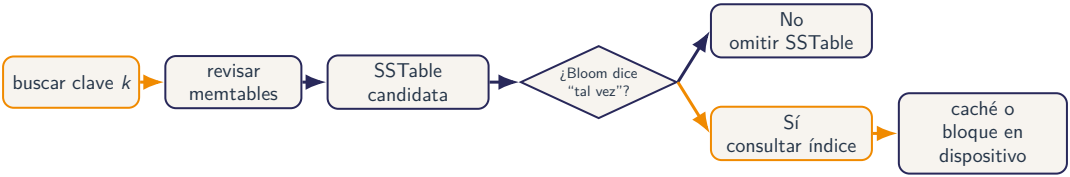
Un **filtro de Bloom** es un resumen probabilístico y compacto de las claves de una SSTable. Para una búsqueda de clave exacta no devuelve el valor: responde **definitivamente ausente** o **posiblemente presente**.



- **Definitivamente ausente:** el motor omite esa SSTable para la búsqueda.
- **Posiblemente presente:** el motor consulta el índice y, si corresponde, los bloques de datos; puede ser un falso positivo.
- Un filtro bien construido no debe producir falsos negativos: no puede descartar un archivo que sí contiene la clave.

Ruta de lectura LSM: descartar antes de leer el dispositivo

NÚCLEO



Orden de búsqueda

Primero se revisa lo más reciente. Para cada SSTable candidata, Bloom puede evitar una lectura física.

Resultado correcto

Si aparecen varias versiones, el motor elige la más reciente que sea visible y respeta las marcas de borrado.

- Un falso positivo agrega trabajo; un falso negativo haría perder un dato existente y no es admisible.
- La caché de bloques evita acceder al dispositivo cuando el bloque requerido ya está en memoria.

Panel de compactación I: qué señales observar

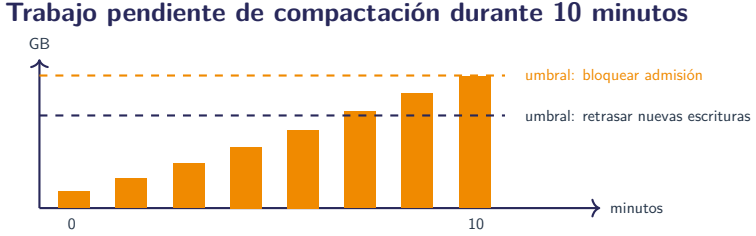
NÚCLEO

Señal	Valor ilustrativo	Qué informa
Ingreso lógico	90 MB/s	Trabajo nuevo recibido por el motor.
Capacidad de compactación	60 MB/s	Ritmo de reorganización de archivos.
Déficit sostenido	30 MB/s	Trabajo que se acumula por segundo.
Trabajo pendiente	18 GB	Datos todavía sin compactar.
Archivos en L0	28 de 36	Cercanía al umbral de protección.
Percentil 99 de escritura	En aumento	Impacto sobre las operaciones más lentas.

Consideración. Se deben relacionar el ingreso, la capacidad de mantenimiento, el crecimiento del trabajo pendiente y la latencia; ninguna cifra aislada basta para diagnosticar.

Panel de compactación II: leer la evolución

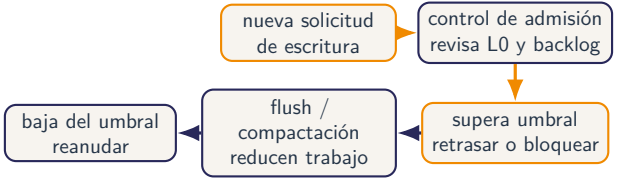
NÚCLEO



Cadena causal

Ingreso mayor que capacidad de compactación ⇒ backlog creciente ⇒ más SSTables en L0 ⇒ más trabajo de lectura y mayor latencia ⇒ regulación de nuevas escrituras.

Cómo regula escrituras un write stall



Implementaciones típicas

- dormir o bloquear el hilo escritor;
- limitar tokens, concurrencia o bytes por segundo;
- esperar a que disminuyan las SSTables L0;
- en algunos sistemas, responder sobrecarga.

Interpretación

La pausa protege una cola finita; no crea capacidad. Si es frecuente o prolongada, deben revisarse carga, hardware y política de compactación.

Tres amplificaciones: medir el trabajo adicional

NÚCLEO

Métrica	Definición	Ejemplo
Amplificación de escritura	Bytes físicos escritos divididos por bytes lógicos aceptados. Incluye WAL, flush, compactación e índices según el alcance medido.	1 GB lógico produce 6 GB físicos: amplificación $= 6$.
Amplificación de lectura	Estructuras o bloques adicionales que deben consultarse para una lectura lógica.	Una clave obliga a revisar 4 SSTables antes de hallar su versión.
Amplificación de espacio	Espacio físico ocupado dividido por el tamaño del estado lógico vivo.	1 GB vivo ocupa 1,5 GB: amplificación $= 1{,}5$.

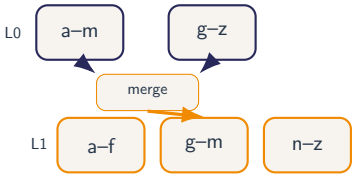
"Amplificación" no significa error: significa que la implementación realiza más trabajo físico que la operación lógica visible para la aplicación.

Compactación por niveles y por grupos: qué cambia

NÚCLEO

Por niveles (leveled)

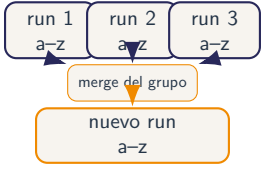
Cada nivel procura rangos poco solapados. Los datos se fusionan varias veces al avanzar hacia niveles mayores.



- menos archivos candidatos por clave;
- mejor control del espacio;
- más reescrituras durante los merges.

Por grupos (tiered / universal)

Se conservan varios runs completos, normalmente solapados, y se fusiona un grupo cuando alcanza cierto tamaño o cantidad.



- menos reescritura inmediata;
- más archivos potenciales por lectura;
- más espacio temporal durante la fusión.

Consulta por clave exacta: busca un identificador concreto, como 9F2A. Si sus percentiles 95 y 99 deben ser bajos, revisar muchos runs resulta costoso.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Integración: decidir con evidencia

Sistemas Distribuidos

Árbol B o LSM-tree: orientación, no ley universal

NÚCLEO

Criterio	Árbol B orientado a páginas	LSM-tree orientada a runs
Escritura	Actualiza páginas de datos e índices; puede dividir páginas.	WAL + memtable + flush; compactación posterior.
Clave exacta	Pocas páginas si el índice coincide con la consulta.	Varias SSTables posibles; Bloom y caché reducen lecturas.
Rangos	Recorrido natural de hojas ordenadas.	Eficiente si hay pocos runs relevantes.
Mantenimiento	Divisiones, VACUUM, bloat y reconstrucción.	Tombstones, compactación, amplificaciones y stalls.
Carga frecuente	Transacciones mixtas; claves exactas y rangos.	Ingestión alta y sostenida; escrituras agrupables.
Pregunta crítica	¿Qué consultas justifican el índice y cuánto encarece escribir?	¿Puede el mantenimiento sostener el ingreso sin backlog creciente?

Criterio

Compare patrón de acceso, garantía, capacidad sostenible, latencia en percentiles 95 y 99 y amplificaciones medidas; no el nombre del producto.

Dos casos cercanos, dos hipótesis defendibles

NÚCLEO

CampusShop

- Transacciones cortas.
- Inventario e idempotencia.
- Consultas por estudiante y fecha.
- Percentil 99 predecible.

Hipótesis: WAL + MVCC + árbol B.

Telemetría del campus

- Ingestión masiva y secuencial.
- Rangos por dispositivo y tiempo.
- Retención temporal.
- Vistas con frescura eventual.

Hipótesis: WAL + LSM + compactación temporal.

Todavía falta evidencia

Medir percentiles 95 y 99, throughput sostenible, objetivo de punto de recuperación (RPO), objetivo de tiempo de recuperación (RTO), amplificación, backlog, tasa de aciertos de caché y espacio.

Repasemos

ACTIVIDAD

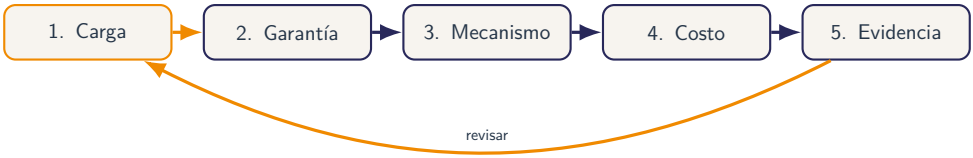
- El COMMIT puede ser durable aunque la página siga atrasada.
- Copiar en memoria principal no equivale a persistir.
- MVCC reduce contención, pero no elimina todas las anomalías.
- Un índice puede no utilizarse y siempre tiene costo de mantenimiento.
- Una LSM desplaza trabajo hacia el flush y la compactación.

Patrón común

Ningún mecanismo elimina el costo: cambia *cuándo*, *dónde* y *cómo* se paga.

Método reusable para decidir almacenamiento

NÚCLEO



- Describa proporciones de lectura y escritura, tamaño, sesgo, crecimiento y consultas.
- Declare qué falla debe tolerarse y qué significa durable en ese modelo.
- Elija un mecanismo inicial, no un producto por reputación.
- Identifique el costo que se desplaza a otra etapa.
- Diseñe un experimento capaz de contradecir su elección.

Cierre: de la intuición a la evidencia

NÚCLEO



Idea final

El almacenamiento no es una caja debajo del sistema. Es la capa donde las promesas lógicas se convierten en bytes, esperas, recuperación y riesgos medibles.

Exit ticket

ACTIVIDAD

En casa, procure responder en tres líneas por punto:

1. ¿Qué diferencia hay entre *aceptado*, *escrito* y *durable*?
2. ¿Qué condición de carrera ilustra el write skew y por qué MVCC no basta para evitarla?
3. ¿Qué diferencia hay entre `EXPLAIN` y `EXPLAIN ANALYZE`?
4. ¿Cómo aplica realmente el motor una pausa de escritura y qué métricas justifican revisarla?

Puente a la clase 7

Hoy vimos cómo un nodo conserva y organiza su historia. La próxima clase estudia cómo un log se convierte en una interfaz distribuida entre productores y consumidores.